FITTRACK: A COLLABORATIVE FITNESS AND NUTRITION PLATFORM WITH PERSISTENT AI HEALTH COACHING MEMORY

Vinod Balasubramaniam and Prem Srinivas
Department of Computer Science and Design
Rajalakshmi Engineering College (Anna University), Chennai, India
Roll Nos.: 241701065, 241701044

Abstract— Digital fitness tools often separate workout tracking, nutrition planning, and AI-based coaching into disconnected experiences. This can make it difficult to retain useful context such as dietary preferences, injury constraints, prior workout records, and goals across sessions, while also coordinating clients and human trainers. This paper presents FitTrack, a browser-based collaborative fitness and nutrition platform designed around persistent health memory and shared workspace updates. The Phase I implementation combines a React 19 and TypeScript frontend, a FastAPI asynchronous backend, WebSocket event streaming, a LiteLLM-based reasoning-and-action coaching layer, and SQLite relational persistence. Optimistic concurrency control (OCC) is used to detect conflicting edits, while atomic upserts maintain deduplicated health-memory records. An automated verification suite of 17 tests passed, covering event contracts, path security, version conflicts, memory persistence, API behavior, and asynchronous terminal execution. The paper summarizes the architecture, implementation, verification results, limitations, and planned Phase II extensions.

Keywords— fitness technology, personalized coaching, persistent health memory, collaborative systems, WebSockets, FastAPI, optimistic concurrency control, AI-assisted wellness

I. INTRODUCTION

Fitness and nutrition applications increasingly provide workout logs, progress dashboards, and automated recommendations. However, these features can operate as separate experiences, leaving important user context—such as injury history, dietary restrictions, and evolving goals—difficult to preserve across repeated coaching sessions. Collaboration introduces another challenge: clients, trainers, and an AI coach may update the same records, and uncoordinated changes can overwrite useful information.

FitTrack is designed as a browser-native, collaborative platform that brings fitness tracking, nutrition-related context, human oversight, and AI coaching into a shared workspace. Its main design goals are to preserve relevant user facts across sessions, support real-time event communication, and protect shared records from silent conflicting writes. Phase I focuses on the full-stack architecture and automated functional verification; wearable integrations and evidence retrieval are planned future work.

The contributions of this work are: (i) a multi-tier architecture for collaborative fitness and nutrition workflows; (ii) a relational memory design for retaining user-specific health context; (iii) version-based optimistic concurrency control with explicit conflict feedback; and (iv) a 17-test automated verification suite for core backend behavior.

II. SYSTEM ARCHITECTURE

A. Architecture Overview

FitTrack uses four connected layers. The presentation layer is implemented with React 19, TypeScript, Vite, Tailwind CSS, and Recharts. The backend gateway uses Python and FastAPI to handle REST endpoints, validate payloads with Pydantic, coordinate asynchronous tasks, and maintain full-duplex WebSocket connections. The AI coaching layer uses LiteLLM to abstract multiple model providers and supports a ReAct-style reasoning-and-action loop. The persistence layer uses SQLite through SQLAlchemy and aiosqlite for relational records and health-memory storage.

B. Real-Time Collaboration

WebSocket events carry structured updates between the browser and backend. Typed event contracts cover presence changes, agent status, tool lifecycle events, file changes, terminal output, and memory updates. Pydantic validation checks incoming payloads, while TypeScript provides compile-time checking on the client. This contract-based design helps reduce schema drift between frontend and backend components.

C. Persistent Health Memory

Conversation history is transient, whereas selected user facts need to remain available for later coaching turns. FitTrack stores these facts in a relational user_memories table using user identifiers, categories, keys, values, and update timestamps. A composite uniqueness constraint on (user_id, key) and SQLite ON CONFLICT upsert behavior allow updated facts to replace older values without creating duplicate rows. A context-building component retrieves relevant records for the AI prompt.

D. Concurrency and Workspace Integrity

Shared records use monotonic version numbers to prevent stale updates from silently replacing newer content. A write succeeds when the expected version matches the current version; the server then saves the update, increments the version, and emits an event. If the versions differ, the server returns HTTP 409 Conflict with the current server content so the client can compare and resolve changes. Path validation also constrains file operations to the designated project workspace.

III. IMPLEMENTATION

The client dashboard provides responsive views, interactive charts, multi-tab workout logging buffers, live multi-user presence, and a side-by-side conflict-resolution interface. FastAPI coordinates application logic, REST endpoints, WebSocket streaming, and asynchronous background operations. The AI layer can invoke supported tools through a ReAct loop, while LiteLLM provides a model-agnostic interface for provider routing. SQLite is configured for write-ahead logging, with SQLAlchemy/aiosqlite supporting asynchronous access.

TABLE I. TECHNOLOGY STACK

Layer	Technology	Purpose
Frontend	React 19, TypeScript, Vite	Responsive dashboard and typed client events
UI / charts	Tailwind CSS, Recharts	Layout and interactive progress visualization
Backend	Python, FastAPI, Pydantic	API, validation, async workflow and WebSockets
AI orchestration	LiteLLM, ReAct pattern	Model abstraction and tool-enabled coaching
Persistence	SQLite, SQLAlchemy, aiosqlite	Relational records, health memory and versioning

A. Core Modules

The Phase I system includes user and profile management, workout logging, persistent memory management, AI coaching orchestration, workspace/file handling, real-time collaboration, and conflict resolution. These modules share the backend gateway and relational store, allowing updates to be reflected across connected clients while retaining a record of current state.

B. Security and Data Handling

The implementation applies runtime schema validation to API and WebSocket payloads and canonical path checks to workspace operations. The automated tests include path traversal attempts, stale-version writes, and memory upsert behavior. These controls provide foundational safeguards, although production deployment would require additional review of authentication, authorization, secret management, isolation, and privacy practices.

IV. TESTING AND RESULTS

The project includes an automated suite executed with pytest and pytest-asyncio. Seventeen test cases passed, giving a 100% pass rate for the specified Phase I test suite. Coverage includes WebSocket event normalization, workspace file creation and path restrictions, optimistic concurrency checks, REST API version handling, persistent health-memory updates, context retrieval, and asynchronous terminal execution.

TABLE II. VERIFICATION SUMMARY

Test area	Verified behavior	Result
Event contracts	Typed WebSocket payloads serialize and validate	Pass
Workspace security	Path traversal outside project root is rejected	Pass
Concurrency	Stale version returns HTTP 409; valid write increments version	Pass
Health memory	Upsert updates existing record without duplication	Pass
API and tools	Endpoint contracts and async terminal output are handled	Pass

These results confirm that the tested workflows behaved as specified in the Phase I environment. They do not establish performance under heavy load, clinical effectiveness, or usability with a broad population of users. Those evaluations remain necessary before the platform can be considered production-ready for health-related use.

V. LIMITATIONS AND FUTURE WORK

Phase I uses file- and record-level concurrency control; simultaneous fine-grained co-editing still requires manual conflict resolution. Context retrieval currently relies on relational facts and deterministic relevance selection rather than a full semantic retrieval pipeline. Host-level subprocess execution is path-restricted but does not yet provide the stronger isolation of disposable containers or microVMs. The platform also does not yet integrate wearable sensors or attendance/clinical systems.

Phase II plans include wearable telemetry integration (for example, Apple HealthKit and Fitbit), hybrid retrieval combining relational profile facts with vector search over exercise-science sources, and isolated execution environments for analytics tasks. Additional priorities include usability studies, load testing, and more comprehensive security evaluation. Any AI-generated fitness guidance should be treated as informational and reviewed appropriately where injuries or medical conditions are involved.

VI. CONCLUSION

FitTrack presents a collaborative fitness and nutrition platform that combines persistent user-specific health memory, real-time event communication, AI coaching orchestration, and optimistic concurrency control. The Phase I implementation connects a React/TypeScript interface with a FastAPI backend, LiteLLM-based orchestration, and SQLite persistence. All 17 automated verification tests passed across the selected functional and security-related scenarios. Future work will focus on wearable integration, evidence-grounded retrieval, stronger execution isolation, and broader evaluation with real users.

REFERENCES

[1] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” International Conference on Learning Representations, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[2] C. Packer et al., “MemGPT: Towards LLMs as Operating Systems,” arXiv:2310.08560, 2023.

[3] J. Cosentino et al., “Towards a Personal Health Large Language Model,” arXiv:2406.06474, 2024. Published as “A personal health large language model for sleep and fitness coaching,” Nature Medicine, 2025, doi:10.1038/s41591-025-03888-0.

[4] A. Metwally et al., “The Anatomy of a Personal Health Agent,” arXiv:2508.20148, 2025.

[5] H. T. Kung and J. T. Robinson, “On Optimistic Methods for Concurrency Control,” ACM Transactions on Database Systems, vol. 6, no. 2, pp. 213–226, 1981.

[6] M. Shapiro et al., “Conflict-Free Replicated Data Types,” in Stabilization, Safety, and Security of Distributed Systems, LNCS 6976, pp. 386–400, 2011.